

Problem 1: Circle Class

Objective:

- To create a Circle class with radius and area
- To initialize multiple circles and calculate their areas and total area

Theory:

A circle is defined by its radius and its area can be calculated using the formula $\text{Area} = \pi * r * r$. OOP allows the radius and area calculation method to be combined into a single class. This makes it easier to manage multiple circles and perform operations on each object separately

Code:

```
Lab 1 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Circle
5  {
6  private:
7      int radius;
8
9  public:
10     Circle(int r) : radius(r) {} // Constructor to initializer list
11
12     float getArea() const
13     {
14         return 3.14159 * radius * radius; // To calculate area
15     }
16 };
17
18 int main()
19 {
20     Circle c1(2), c2(3), c3(5);
21
22     float total = c1.getArea() + c2.getArea() + c3.getArea();
23
24     cout << "Area of circle 1: " << c1.getArea() << endl;
25     cout << "Area of circle 2: " << c2.getArea() << endl;
26     cout << "Area of circle 3: " << c3.getArea() << endl;
27     cout << "Total area of three circles: " << total << endl;
28
29     return 0;
30 }
```

Procedure:

1. The Circle class was created with radius as a private member
2. A method to calculate area was written
3. Three Circle objects were initialized in main
4. Individual areas and total area were calculated and displayed

Output:

```
Area of circle 1: 12.5664
Area of circle 2: 28.2743
Area of circle 3: 78.5397
Total area of three circles: 119.38
```

Precautions:

- Radius should be positive
- Formula should be correctly implemented

Conclusion:

The area of each circle and the total area were successfully calculated using the Circle class. Object-oriented programming was demonstrated effectively by handling multiple circle objects with their own data and methods

Problem 2: Account Class

Objective:

- To create an Account class to simulate banking operations
- To perform deposit, withdrawal, transfer and display balances

Theory:

A bank account has an account number and a balance. Deposit increases the balance, withdrawal decreases it and transfer moves money from one account to another

Code:

```
Lab 1 > C++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Account
5  {
6  private:
7      int number;
8      int amount;
9
10 public:
11     // Constructor to initialize list
12     Account(int num, int amt) : number(num), amount(amt) {}
13
14     // To deposit money
15     void deposit(int x)
16     {
17         amount += x;
18         cout << "Deposited " << x << " to Account " << number << endl;
19     }
20
21     // To withdraw money with balance check
22     void withdraw(int x)
23     {
24         if (x <= amount)
25         {
26             amount -= x;
27             cout << "Withdrew " << x << " from Account " << number << endl;
28         }
29         else
30         {
31             cout << "Insufficient balance in Account " << number << endl;
32         }
33     }
34
35     // To transfer money to another account
36     void transfer(Account &to, int x)
37     {
38         if (x <= amount)
39         {
40             amount -= x;
41             to.amount += x;
42             cout << "Transferred " << x << " from Account " << number
43                 << " to Account " << to.number << endl;
44         }
45         else
46         {
47             cout << "Insufficient balance in Account " << number
48                 << " for transfer" << endl;
49         }
50     }
}
```

```

51
52 // To display account details
53 void show() const
54 {
55     cout << "Account No: " << number << " | Balance: " << amount << endl;
56 }
57 };
58
59 int main()
60 {
61     // To initialize 5 accounts
62     Account a1(2403176, 20);
63     Account a2(2403177, 3000);
64     Account a3(2403178, 7000);
65     Account a4(2403179, 2000);
66     Account a5(2403180, 10000);
67
68     cout << "Initial Account Balances:\n";
69     a1.show();
70     a2.show();
71     a3.show();
72     a4.show();
73     a5.show();
74
75     // To perform some transactions
76     a1.deposit(1000);
77     a2.withdraw(200);
78     a3.transfer(a4, 500);
79
80     cout << "Final Account Balances:\n";
81     a1.show();
82     a2.show();
83     a3.show();
84     a4.show();
85     a5.show();
86
87     return 0;
88 }

```

Procedure:

1. The Account class was created with account number and balance
2. Deposit, withdrawal, transfer and display methods were implemented
3. Accounts were initialized in main
4. Transactions were performed and final balances were displayed

Output:

```

Initial Account Balances:
Account No: 2403176 | Balance: 20
Account No: 2403177 | Balance: 3000
Account No: 2403178 | Balance: 7000
Account No: 2403179 | Balance: 2000
Account No: 2403180 | Balance: 10000
Deposited 1000 to Account 2403176
Withdrawn 200 from Account 2403177
Transferred 500 from Account 2403178 to Account 2403179
Final Account Balances:
Account No: 2403176 | Balance: 1020
Account No: 2403177 | Balance: 2800
Account No: 2403178 | Balance: 6500
Account No: 2403179 | Balance: 2500
Account No: 2403180 | Balance: 10000

```

Precautions:

- Ensure withdrawal or transfer amount $\leq$ balance
- IDs should be unique

Conclusion:

Deposits, withdrawals and transfers were performed successfully. The Account class allowed secure and organized management of balances, demonstrating object-oriented principles in a simple banking system

Problem 3: Triangle Class

Objective:

- To create a Triangle class
- To initialize edges, check validity and calculate the area

Theory:

A triangle is defined by its three edges. A triangle is valid if the sum of any two edges is greater than the third edge. The area can be calculated using the formula: $s = (a+b+c)/2$ and $\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$. Using a class allows both the edges and the methods for validation and area calculation to be combined in one structure

Code:

```
Lab 1 > C++ 3.cpp > main()
1  #include <iostream>
2  #include <cmath> // for sqrt
3  using namespace std;
4
5  class Triangle
6  {
7  private:
8      int edge1, edge2, edge3;
9      float area;
10
11 public:
12     // Constructor to initialize edges
13     Triangle(int e1, int e2, int e3) : edge1(e1), edge2(e2), edge3(e3), area(0) {}
14
15     // To check if the edges can form a valid triangle
16     bool isValid() const
17     {
18         return (edge1 + edge2 > edge3) &&
19             (edge1 + edge3 > edge2) &&
20             (edge2 + edge3 > edge1);
21     }
22
23     // To calculate area
24     void calculateArea()
25     {
26         if (isValid())
27         {
28             float s = (edge1 + edge2 + edge3) / 2.0;
29             area = sqrt(s * (s - edge1) * (s - edge2) * (s - edge3));
30         }
31         else
32         {
33             area = 0;
34         }
35     }
36
37     // To display the area
38     void showArea() const
39     {
40         if (area > 0)
41             cout << "Area of the triangle: " << area << endl;
42         else
43             cout << "Cannot calculate area. Invalid triangle edges." << endl;
44     }
45
46     // To display the edges
47     void showEdges() const
48     {
49         cout << "Edges: " << edge1 << ", " << edge2 << ", " << edge3 << endl;
50     }
51 };
```

```

52
53 int main()
54 {
55     // To initialize a triangle
56     Triangle t(3, 4, 5);
57
58     cout << "Triangle edges:\n";
59     t.showEdges();
60
61     // To check validity
62     if (t.isValid())
63     {
64         cout << "The edges form a valid triangle.\n";
65         t.calculateArea();
66         t.showArea();
67     }
68     else
69     {
70         cout << "The edges do NOT form a valid triangle.\n";
71     }
72
73     return 0;
74 }

```

Procedure:

1. The Triangle class was created with three edges as private members
2. Validity was checked using the triangle inequality rule
3. Area was calculated
4. The area was displayed for valid triangles

Output:

```

Triangle edges:
Edges: 3, 4, 5
The edges form a valid triangle.
Area of the triangle: 6

```

Precautions:

- Edges must be positive
- Check validity before calculating area

Conclusion:

The Triangle class successfully verified triangle validity and calculated its area. Object-oriented principles allowed encapsulation of edges and operations efficiently

Problem 4: Gym Member

Objective:

- To create a Member class for gym members
- To calculate BMI and classify members' health status
- To find min, max and average height & weight

Theory:

Each member has an ID, height and weight. BMI is calculated using the formula $BMI = \text{Weight} / \text{Height}^2$ and the member is classified as underweight, normal, overweight, or obese according to standard BMI ranges. Classes make it easy to handle multiple members and perform operations like min, max, average and BMI classification

Code:

```
Lab 1 > C++ 4.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <limits>
4  using namespace std;
5
6  // Structure to store Member data
7  struct Member {
8      int id;
9      float height; // meters
10     float weight; // kg
11
12     // To calculate BMI
13     float calculateBMI() const {
14         return weight / (height * height);
15     }
16
17     // To display BMI
18     void displayBMI() const {
19         float bmi = calculateBMI();
20         cout << "Member ID: " << id << " | BMI: " << bmi << " | Classification: ";
21         if (bmi < 16) cout << "Severe Thinness";
22         else if (bmi < 17) cout << "Moderate Thinness";
23         else if (bmi < 18.5) cout << "Mild Thinness";
24         else if (bmi < 25) cout << "Normal";
25         else if (bmi < 30) cout << "Overweight";
26         else if (bmi < 35) cout << "Obese Class I";
27         else if (bmi < 40) cout << "Obese Class II";
28         else cout << "Obese Class III";
29         cout << endl;
30     }
31 };
32
33 // Global vector to store members
34 vector<Member> members;
35
36 // Function to add a member
37 void addMember() {
38     Member m;
39     cout << "Enter Member ID: ";
40     cin >> m.id;
41     cout << "Enter Height (m): ";
42     cin >> m.height;
43     cout << "Enter Weight (kg): ";
44     cin >> m.weight;
45     members.push_back(m);
46     cout << "Member added successfully.\n";
47 }
48
```

```

49 // Function to update a member
50 void updateMember() {
51     int id;
52     cout << "Enter Member ID to update: ";
53     cin >> id;
54     for (auto &m : members) {
55         if (m.id == id) {
56             cout << "Enter new Height (m): ";
57             cin >> m.height;
58             cout << "Enter new Weight (kg): ";
59             cin >> m.weight;
60             cout << "Member updated successfully.\n";
61             return;
62         }
63     }
64     cout << "Member not found.\n";
65 }
66
67 // Function to remove a member
68 void removeMember() {
69     int id;
70     cout << "Enter Member ID to remove: ";
71     cin >> id;
72     for (auto it = members.begin(); it != members.end(); ++it) {
73         if (*it->id == id) {
74             members.erase(it);
75             cout << "Member removed successfully.\n";
76             return;
77         }
78     }
79     cout << "Member not found.\n";
80 }
81
82 // Function to calculate max height and weight
83 void maxHeightWeight() {
84     if (members.empty()) {
85         cout << "No members available.\n";
86         return;
87     }
88     float maxH = members[0].height;
89     float maxW = members[0].weight;
90     for (const auto &m : members) {
91         if (m.height > maxH) maxH = m.height;
92         if (m.weight > maxW) maxW = m.weight;
93     }
94     cout << "Max Height: " << maxH << " m | Max Weight: " << maxW << " kg\n";
95 }
96
97 // Function to calculate min height and weight
98 void minHeightWeight() {
99     if (members.empty()) {
100         cout << "No members available.\n";
101         return;
102     }
103     float minH = members[0].height;
104     float minW = members[0].weight;
105     for (const auto &m : members) {
106         if (m.height < minH) minH = m.height;
107         if (m.weight < minW) minW = m.weight;
108     }
109     cout << "Min Height: " << minH << " m | Min Weight: " << minW << " kg\n";
110 }
111
112 // Function to calculate average height and weight
113 void averageHeightWeight() {
114     if (members.empty()) {
115         cout << "No members available.\n";
116         return;
117     }
118     float sumH = 0, sumW = 0;
119     for (const auto &m : members) {
120         sumH += m.height;
121         sumW += m.weight;
122     }
123     cout << "Average Height: " << sumH / members.size() << " m | Average Weight: " << sumW / members.size() << " kg\n";
124 }
125
126 // Function to display BMI of all members
127 void displayBMI() {
128     if (members.empty()) {
129         cout << "No members available.\n";
130         return;
131     }
132     for (const auto &m : members) {
133         m.displayBMI();
134     }
135 }
136

```

```

137 int main() {
138     int choice;
139     do {
140         cout << "\n***** Main Menu *****\n";
141         cout << "1. Add Member\n";
142         cout << "2. Update Member\n";
143         cout << "3. Remove Member\n";
144         cout << "4. Max Height and Weight\n";
145         cout << "5. Min Height and Weight\n";
146         cout << "6. Average Height and Weight\n";
147         cout << "7. BMI Classification\n";
148         cout << "8. Exit\n";
149         cout << "Enter your option (1-8): ";
150         cin >> choice;
151
152         switch (choice) {
153             case 1: addMember(); break;
154             case 2: updateMember(); break;
155             case 3: removeMember(); break;
156             case 4: maxHeightWeight(); break;
157             case 5: minHeightWeight(); break;
158             case 6: averageHeightWeight(); break;
159             case 7: displayBMI(); break;
160             case 8: cout << "Exiting program.\n"; break;
161             default: cout << "Invalid option. Try again.\n";
162         }
163     } while (choice != 8);
164
165     return 0;
166 }

```

Procedure:

1. The Member class was created with ID, height and weight
2. Members were added, updated and removed from the system
3. BMI was calculated and classification was displayed
4. Min, max and average height & weight were calculated and displayed

Output:

```

***** Main Menu *****
1. Add Member
2. Update Member
3. Remove Member
4. Max Height and Weight
5. Min Height and Weight
6. Average Height and Weight
7. BMI Classification
8. Exit
Enter your option (1-8): █

```

Precautions:

- Enter positive height and weight
- IDs must be unique

Conclusion:

BMI calculations and member management were successfully implemented. Object-oriented programming was used to organize multiple members efficiently and statistical operations were applied accurately

Problem 1: Test Class

Objective:

- To create a Test class with multiple constructors
- To track the total number of objects created
- To perform operations using object data members

Theory:

In object-oriented programming, constructors are used to initialize objects in different ways such as default, parameterized and copy constructors. A static data member is shared among all objects of a class and is useful for counting how many objects are created. Encapsulation ensures that data members remain protected while methods control access to them

Code:

```
Lab 2 > C++ P1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Test {
5  private:
6      int x, y;
7      static int z;    // To count total objects
8
9  public:
10     // Empty constructor
11     Test() {
12         x = 0;
13         y = 0;
14         z++;
15     }
16
17     // Parameterized constructor
18     Test(int a, int b) {
19         x = a;
20         y = b;
21         z++;
22     }
23
24     // Copy constructor
25     Test(const Test &t) {
26         x = t.x;
27         y = t.y;
28         z++;
29     }
30
31     // Method to initialize x and y
32     void setData(int a, int b) {
33         x = a;
34         y = b;
35     }
36
37     // To display only z
38     static void showZ() {
39         cout << "Total objects: " << z << endl;
40     }
41 }
```

```

42 // To display x, y, z
43 void showData() const {
44     cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
45 }
46
47 int getX() const { return x; }
48 int getY() const { return y; }
49
50 // Destructor
51 ~Test() {}
52 };
53
54 int Test::z = 0;
55
56 int main() {
57     Test t1, t2(3,4), t3(t2), t4(5,6), t5(1,9);
58
59     Test::showZ();
60
61     int sumX = t1.getX() + t2.getX() + t3.getX() + t4.getX() + t5.getX();
62     cout << "Sum of x: " << sumX << endl;
63
64     Test arr[5] = {t1, t2, t3, t4, t5};
65     int maxY = arr[0].getY(), index = 0;
66
67     for(int i=1; i<5; i++) {
68         if(arr[i].getY() > maxY) {
69             maxY = arr[i].getY();
70             index = i;
71         }
72     }
73
74     cout << "Object number with max y: " << index+1 << endl;
75     return 0;
76 }

```

Procedure:

1. The Test class was created with private data members x, y and static z
2. Constructors were implemented to initialize data in different ways
3. Five objects were created using different constructors
4. The sum of x values and maximum y value were calculated
5. Destructor was defined for object cleanup

Output:

```

Total objects: 5
Sum of x: 12
Object number with max y: 5

```

Precautions:

- Static variable must be defined outside the class
- Copy constructor should receive object by reference
- Constant methods should not modify data members

Conclusion:

The Test class successfully demonstrated the use of constructors, static data members and object operations. Object counting and data analysis were performed correctly, showing proper use of object-oriented programming concepts

Problem 2: Copy Constructor

Objective:

- To create a class with two data members
- To use setter method and copy constructor

Theory:

A copy constructor is used to create a new object by copying data from an existing object. Setter methods help initialize private data members safely. This approach ensures controlled access and proper duplication of object data

Code:

```
Lab 2 > C++ P2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int a, b;
7
8  public:
9      void setData(int x, int y) {
10         a = x;
11         b = y;
12     }
13
14     // Copy constructor
15     A(const A &obj) {
16         a = obj.a;
17         b = obj.b;
18     }
19
20     A() {}
21
22     void show() const {
23         cout << "a = " << a << ", b = " << b << endl;
24     }
25 };
26
27 int main() {
28     A obj1;
29     obj1.setData(10, 20);
30
31     A obj2(obj1); // Copy constructor call
32
33     obj1.show();
34     obj2.show();
35
36     return 0;
37 }
```

Procedure:

1. The class A was created with two private data members
2. Setter method was used to assign values to obj1
3. Copy constructor was used to copy data from obj1 to obj2
4. Values were displayed for verification

Output:

```
a = 10, b = 20
a = 10, b = 20
```

Precautions:

- Copy constructor must accept reference parameter
- Setter method should be used before copying data
- Private members should not be accessed directly

Conclusion:

The program successfully demonstrated the use of setter methods and copy constructors. Object data was copied correctly, ensuring safe and structured duplication of objects

Problem 1: Father → Son → GrandSon Inheritance

Objective:

- To create a multi-level inheritance hierarchy: Father → Son → GrandSon
- To explore access to private, protected and public members in derived classes
- To calculate the sum of accessible members in GrandSon class

Theory:

Inheritance allows a class to acquire members and methods from another class. Access specifiers (private, protected, public) control which members are accessible in derived classes. Multi-level inheritance demonstrates how access modifiers affect visibility across different levels of hierarchy. By testing various inheritance types, one can see which members are accessible in Son and GrandSon classes and how sums of accessible members can be calculated

Code:

```
Lab 3 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  // Father class
5  class Father {
6  private:
7      int money;
8  protected:
9      int gold;
10 public:
11     int land;
12
13 public:
14     Father() { money = 100; gold = 50; land = 200; } // To initialize values
15 };
16 // Son class inheriting from Father
17 class Son : public Father { // To change public into protected/private
18 public:
19     void showSon() {
20         // money; // Not accessible
21         cout << "gold in Son: " << gold << endl; // Accessible
22         cout << "land in Son: " << land << endl; // Accessible
23     }
24 };
25 // GrandSon class inheriting from Son
26 class GrandSon : public Son { // To change public into protected/private
27 public:
28     void showGrandSon() {
29         // money; // Not accessible
30         cout << "gold in GrandSon: " << gold << endl;
31         cout << "land in GrandSon: " << land << endl;
32         int sum = gold + land; // money not accessible
33         cout << "Sum of accessible members: " << sum << endl;
34     }
35 };
36 int main() {
37     Son s;
38     GrandSon gs;
39     s.showSon();
40     gs.showGrandSon();
41     return 0;
42 }
```

Procedure:

1. The Father class was created with private, protected and public members
2. The Son class was derived from Father with public inheritance
3. The GrandSon class was derived from Son with public inheritance
4. Accessible members in Son and GrandSon classes were tested and displayed
5. Sum of accessible members in GrandSon class was calculated
6. Different inheritance types were tested by modifying access specifiers

Output:

```
gold in Son: 50
land in Son: 200
gold in GrandSon: 50
land in GrandSon: 200
Sum of accessible members: 250
```

- **Son class:** gold and land accessible; money not accessible
- **GrandSon class:** gold and land accessible; money not accessible
- Sum of accessible members in GrandSon class displayed correctly

Class		In Son Class			In GrandSon Class		
Son	GrandSon	money	gold	land	money	gold	land
public	public	×	✓	✓	×	✓	✓
protected	public	×	✓	×	×	✓	×
private	public	×	×	×	×	×	×
public	protected	×	✓	✓	×	✓	×
protected	protected	×	✓	×	×	✓	×
private	protected	×	×	×	×	×	×
public	private	×	✓	✓	×	×	×
protected	private	×	✓	×	×	×	×
private	private	×	×	×	×	×	×

Precautions:

- Private members cannot be accessed in derived classes
- Correct inheritance type must be used based on accessibility requirements
- Display functions should only access allowed members

Conclusion:

The problem demonstrates multi-level inheritance and the effect of different access specifiers. Son and GrandSon classes accessed only allowed members of Father and the sum of accessible members was calculated correctly

Problem 2: Different Types of Inheritance

Objective:

- To learn and test different types of inheritance in C++
- To access private, protected and public members from derived classes
- To understand visibility rules for each inheritance type

Theory:

Inheritance in C++ allows one class to derive properties and behaviors from another class. There are multiple types:

1. **Single Inheritance:** One base, one derived class
2. **Multi-Level Inheritance:** Chain of inheritance (base → derived → derived)
3. **Multiple Inheritance:** One derived class inherits from multiple base classes
4. **Hierarchical Inheritance:** Multiple derived classes inherit from the same base
5. **Hybrid Inheritance:** Combination of more than one type of inheritance

Access specifiers (private, protected, public) control which members are visible in derived classes

Codes:

1. Single Inheritance

```
Lab 3 > C++ 2.1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {
16 public:
17     void show() {
18         // cout << x; // Not accessible
19         cout << "y: " << y << endl; // Accessible
20         cout << "z: " << z << endl; // Accessible
21     }
22 };
23
24 int main() {
25     B b;
26     b.show();
27     return 0;
28 }
```

Output:

y: 20
z: 30

2. Multi-Level Inheritance

```
Lab 3 > C++ 2.2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {};
16
17 class C : public B {
18 public:
19     void show() {
20         // cout << x; // Not accessible
21         cout << "y: " << y << endl; // Accessible
22         cout << "z: " << z << endl; // Accessible
23     }
24 };
25
26 int main() {
27     C c;
28     c.show();
29     return 0;
30 }
```

Output:

y: 20
z: 30

3. Multiple Inheritance

```
Lab 3 > C++ 2.3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B {
16 private:
17     int p;
18 protected:
19     int q;
20 public:
21     int r;
22     public:
23     B() { p=5; q=15; r=25; }
24 };
25
26 class C : public A, public B {
27 public:
28     void show() {
29         // cout << x; // Not accessible
30         cout << "y: " << y << endl; // Accessible
31         cout << "z: " << z << endl; // Accessible
32         // cout << p; // Not accessible
33         cout << "q: " << q << endl; // Accessible
34         cout << "r: " << r << endl; // Accessible
35     }
36 };
37
38 int main() {
39     C c;
40     c.show();
41     return 0;
42 }
```


Output:

y: 20
z: 30
q: 15
r: 25

4. Hierarchical Inheritance

```
Lab 3 > C++ 2.4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {
16 public:
17     void show() {
18         // cout << x; // Not accessible
19         cout << "y: " << y << endl;
20         cout << "z: " << z << endl;
21     }
22 };
23
24 class C : public A {
25 public:
26     void show() {
27         // cout << x; // Not accessible
28         cout << "y: " << y << endl;
29         cout << "z: " << z << endl;
30     }
31 };
32
33 int main() {
34     B b;
35     C c;
36     b.show();
37     c.show();
38     return 0;
39 }
```

Output:

y: 20
z: 30
y: 20
z: 30

5. Hybrid Inheritance

```
Lab 3 > C++ 2.5.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int x;
7  protected:
8      int y;
9  public:
10     int z;
11     public:
12     A() { x=10; y=20; z=30; }
13 };
14
15 class B : public A {};
16 class C : public A {};
17
18 class D : public B, public C {
19 public:
20     void show() {
21         // cout << x; // Not accessible
22         cout << "B::y: " << B::y << endl;
23         cout << "B::z: " << B::z << endl;
24         cout << "C::y: " << C::y << endl;
25         cout << "C::z: " << C::z << endl;
26     }
27 };
28
29 int main() {
30     D d;
31     d.show();
32     return 0;
33 }
```

Output:

```
B::y: 20  
B::z: 30  
C::y: 20  
C::z: 30
```

Procedure:

1. Classes were created according to the inheritance type
2. Derived class methods were written to access base class members
3. main() called the methods to observe accessible members
4. Each inheritance type was tested separately
5. Output: was noted for private, protected and public members

Observations:

- Private members of base classes were never accessible in derived classes
- Protected members were accessible in derived classes depending on inheritance type
- Public members were accessible in public inheritance
- Different inheritance types demonstrated different member accessibilities

Precautions

- Private members cannot be accessed directly in derived classes
- Correct inheritance type must be used for expected access
- Display functions should not try to access inaccessible members

Conclusion

The problem successfully demonstrated all five types of inheritance in C++. Access specifiers correctly controlled visibility in derived classes. Multi-level, multiple, hierarchical and hybrid inheritance types were successfully tested, showing how inheritance type and access specifiers influence accessibility

Problem 3: Constructors & Destructors in Inheritance

Objective:

- To understand constructor and destructor execution in different types of inheritance.
- To implement constructors and destructors for single, multi-level, multiple, hierarchical and hybrid inheritance.
- To observe and verify the sequence of constructor and destructor calls.

Theory:

In C++, constructors are executed from base class to derived class, while destructors are executed from derived class to base class

1. Single Inheritance

```
Lab 3 > C++ 3.1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     void sum() { cout << getAx() + bx << endl; } // Sum method
20 };
21
22 int main() {
23     B b;
24     b.sum();
25     return 0;
26 }
```

Sequence:

Constructor: $A \rightarrow B$

Destructor: $B \rightarrow A$

2. Multi-Level Inheritance

```
Lab 3 > C++ 3.2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : public B {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + getBx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     C c;
33     c.sum();
34     return 0;
35 }
```

Sequence:

Constructor: $A \rightarrow B \rightarrow C$

Destructor: $C \rightarrow B \rightarrow A$

3. Multiple Inheritance

```
Lab 3 > C++ 3.3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : public A, public B {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + getBx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     C c;
33     c.sum();
34     return 0;
35 }
```

Sequence:

Constructor: $A \rightarrow B \rightarrow C$

Destructor: $C \rightarrow B \rightarrow A$

4. Hierarchical Inheritance

```
Lab 3 > C++ 3.4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     void sum() { cout << getAx() + bx << endl; } // Sum method
20 };
21
22 class C : public A {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     void sum() { cout << getAx() + cx << endl; } // Sum method
29 };
30
31 int main() {
32     B b;
33     b.sum();
34     C c;
35     c.sum();
36     return 0;
37 }
```

Sequence:

For B: Constructor $A \rightarrow B$, Destructor $B \rightarrow A$

For C: Constructor $A \rightarrow C$, Destructor $C \rightarrow A$

5. Hybrid Inheritance

```
Lab 3 > C++ 3.5.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  private:
6      int ax;
7  public:
8      A() { ax = 10; } // Constructor
9      virtual ~A() { } // Destructor
10     int getAx() { return ax; }
11 };
12
13 class B : virtual public A {
14 private:
15     int bx;
16 public:
17     B() { bx = 20; } // Constructor
18     ~B() { } // Destructor
19     int getBx() { return bx; }
20 };
21
22 class C : virtual public A {
23 private:
24     int cx;
25 public:
26     C() { cx = 30; } // Constructor
27     ~C() { } // Destructor
28     int getCx() { return cx; }
29 };
30
31 class D : public B, public C {
32 private:
33     int dx;
34 public:
35     D() { dx = 40; } // Constructor
36     ~D() { } // Destructor
37     void sum() { cout << getAx() + getBx() + getCx() + dx << endl; } // Sum method
38 };
39
40 int main() {
41     D d;
42     d.sum();
43     return 0;
44 }
```

Sequence:

Constructor: $A \rightarrow B \rightarrow C \rightarrow D$

Destructor: $D \rightarrow C \rightarrow B \rightarrow A$

Procedure:

1. Classes were created with private members and constructors/destructors
2. Derived classes implemented their own constructors and destructors
3. Methods were written to calculate the sum of all relevant members
4. `main()` objects were created to observe constructor/destructor sequences
5. Execution order of constructors and destructors was noted for each inheritance type

Output:

- Constructors always executed from base $\rightarrow$ derived
- Destructors executed in reverse: derived $\rightarrow$ base

- Virtual inheritance ensured single construction of shared base class in diamond problem
- Sum methods returned correct results using accessible members

Precautions:

- Always initialize base members properly
- In multiple inheritance, constructors for each base must be correctly called
- For diamond inheritance, use virtual to prevent multiple base construction

Conclusion:

Constructors and destructors in inheritance follow strict sequences: base to derived for construction and derived to base for destruction. This problem demonstrated constructor/destructor execution clearly for single, multi-level, multiple, hierarchical and hybrid inheritance types

Problem 1: Method / Function Overloading

Objective:

- To implement function overloading using a single method name
- To perform summation of different types and numbers of values

Theory:

Function overloading allows multiple functions to have the same name but different parameter lists. The compiler decides which function to call based on the number and type of arguments passed. This improves code readability and flexibility by using one logical function name for related operations

Code:

```
Lab 4 > C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Test {
5  public:
6      int Sum(int a) {
7          return a; // One integer
8      }
9      int Sum(int a, int b) {
10         return a + b; // Two integers
11     }
12     double Sum(double a, int b) {
13         return a + b; // Double and integer
14     }
15     double Sum(int a, double b) {
16         return a + b; // Integer and double
17     }
18     int Sum(double a, double b) {
19         return a + b; // Two double
20     }
21 };
22
23 int main() {
24     Test t;
25     // Calls to overloaded Sum() methods
26     cout << t.Sum(10) << endl;
27     cout << t.Sum(10,20) << endl;
28     cout << t.Sum(5.7,20) << endl;
29     cout << t.Sum(10,2.6) << endl;
30     cout << t.Sum(10.5,20.5) << endl;
31
32     return 0;
33 }
```

Procedure:

1. A Test class was created

2. The Sum() method was overloaded with different parameter types
3. Function calls were made from main()
4. Returned values were displayed

Output:

```
10
30
25.7
12.6
31
```

- Sum(10) → 10
- Sum(10,20) → 30
- Sum(5.7,20) → 25.7
- Sum(10,2.6) → 12.6
- Sum(10.5,20.5) → 31

Precautions:

- Parameter types must differ clearly
- Return type alone should not differ
- Proper data types must be used

Conclusion:

Function overloading was implemented successfully. The program correctly selected the appropriate Sum() method based on function arguments, demonstrating compile time polymorphism

Problem 2: Operator Overloading

Objective:

- To calculate current in an AC circuit using operator overloading
- To represent complex numbers using a class

Theory:

In AC circuits, impedance and voltage are represented as complex numbers. Operator overloading allows arithmetic operations on user defined objects like complex numbers. This simplifies calculations and makes the code more intuitive and readable

Code:

```
Lab 4 > C++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Circuit {
5  private:
6      float real;
7      float img;
8
9  public:
10     // Constructor to initialize real and imaginary
11     Circuit(float r = 0, float i = 0) {
12         real = r;
13         img = i;
14     }
15     // Operator overloading for addition
16     Circuit operator+(Circuit c) {
17         return Circuit(real + c.real, img + c.img);
18     }
19     // Operator overloading for division
20     Circuit operator/(Circuit c) {
21         float d = c.real*c.real + c.img*c.img;
22         return Circuit(
23             (real*c.real + img*c.img)/d,
24             (img*c.real - real*c.img)/d
25         );
26     }
27     // To display real and imaginary values
28     void display() {
29         cout << real << " + j" << img << endl;
30     }
31 };
32
33 int main() {
34     // To initialize impedance values
35     Circuit z1(3,4), z2(4,-3), z3(0,6);
36     // Input voltage
37     Circuit v(100,50);
38     // Current calculation
39     Circuit i = v / (z1 + z2 + z3);
40     i.display();
41     return 0;
42 }
```

Procedure:

1. The Circuit class was created to represent complex numbers
2. Operators were overloaded for addition and division
3. Impedances and voltage were initialized
4. Current was calculated and displayed

Output:

10.7143 + j-3.57143

- Complex arithmetic was performed correctly
- Current value was obtained using overloaded operators

Precautions:

- Division by zero must be avoided
- Correct formula for complex division should be used

Conclusion:

Operator overloading simplified complex number calculations. The circuit current was computed correctly, demonstrating effective use of operator overloading

Problem 3: Unary Operator Overloading

Objective:

- To control TV sound using unary operator overloading.
- To increase and decrease sound using operators.

Theory:

Unary operator overloading allows operators like ++ and -- to be redefined for user-defined classes. This is useful for controlling incremental changes such as sound level in a TV remote system.

Code:

```
Lab 4 > C++ 3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Remote {
5  private:
6      int sound;
7
8  public:
9      Remote() {
10         sound = 0;
11     } // To initialize sound value
12
13     void operator++() {
14         sound++;
15     } // To increase sound
16
17     void operator--() {
18         sound--;
19     } // To decrease sound
20
21     void display() {
22         cout << sound << endl;
23     } // To display sound value
24 };
25
26 int main() {
27     Remote rs;
28     ++rs; // Increase
29     ++rs;
30     --rs; // Decrease
31     rs.display();
32     return 0;
33 }
```

Procedure:

1. A Remote class was created
2. Unary operators ++ and -- were overloaded

3. Sound level was increased and decreased
4. Final sound value was displayed

Output:

1

- Sound increased on ++ operation
- Sound decreased on -- operation
- Final sound level displayed correctly

Precautions:

- Sound should not exceed limits
- Initial sound value should be defined

Conclusion:

In this problem, Unary operator overloading was implemented successfully to control the sound levels of a TV remote. Each operator (++ and --) correctly modified the private sound data member as intended, simulating the increase and decrease of volume. The program demonstrated how unary operators can be customized for user-defined classes, and the final output verified that the remote responded accurately to multiple button presses. This exercise reinforced the concept of operator overloading and showed its practical application in modeling real-world devices

Problem 4: Friend Function

Objective:

- To calculate total money using friend function
- To access private data members of two classes

Theory:

Friend functions are special functions in C++ that are granted access to the private and protected members of one or more classes. This capability is particularly useful when operations require combining or manipulating data from multiple classes while still preserving the principles of encapsulation and data hiding, allowing controlled interaction between class internals without exposing them publicly

Code:

```
Lab 4 > C++ 4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Bank2;
5
6  class Bank1 {
7  private:
8      float money1;
9
10 public:
11     // Constructor to initialize money1
12     Bank1(float m) {
13         money1 = m;
14     }
15
16     // Friend function declaration
17     friend void Sum(Bank1, Bank2);
18 };
19
20 class Bank2 {
21 private:
22     float money2;
23
24 public:
25     // Constructor to initialize money2
26     Bank2(float m) {
27         money2 = m;
28     }
29
30     // Friend function declaration
31     friend void Sum(Bank1, Bank2);
32 };
33
34 // Friend function to calculate total money
35 void Sum(Bank1 b1, Bank2 b2) {
36     cout << b1.money1 + b2.money2 << endl;
37 }
38
39 int main() {
40     Bank1 b1(2000);
41     Bank2 b2(4000);
42
43     Sum(b1, b2);
44     return 0;
45 }
```

Procedure:

1. Two bank classes were created with private money variables
2. A friend function was declared and defined
3. Total money was calculated and displayed

Output: 6000

- Correct total money was displayed
- Private data was accessed successfully

Precautions:

- Friend functions should be used carefully
- Class relationships should be clear

Conclusion:

The friend function successfully accessed private data members from two classes. Total money calculation was performed correctly, demonstrating controlled data sharing